LAB # 6

Name: Hifza Hussain

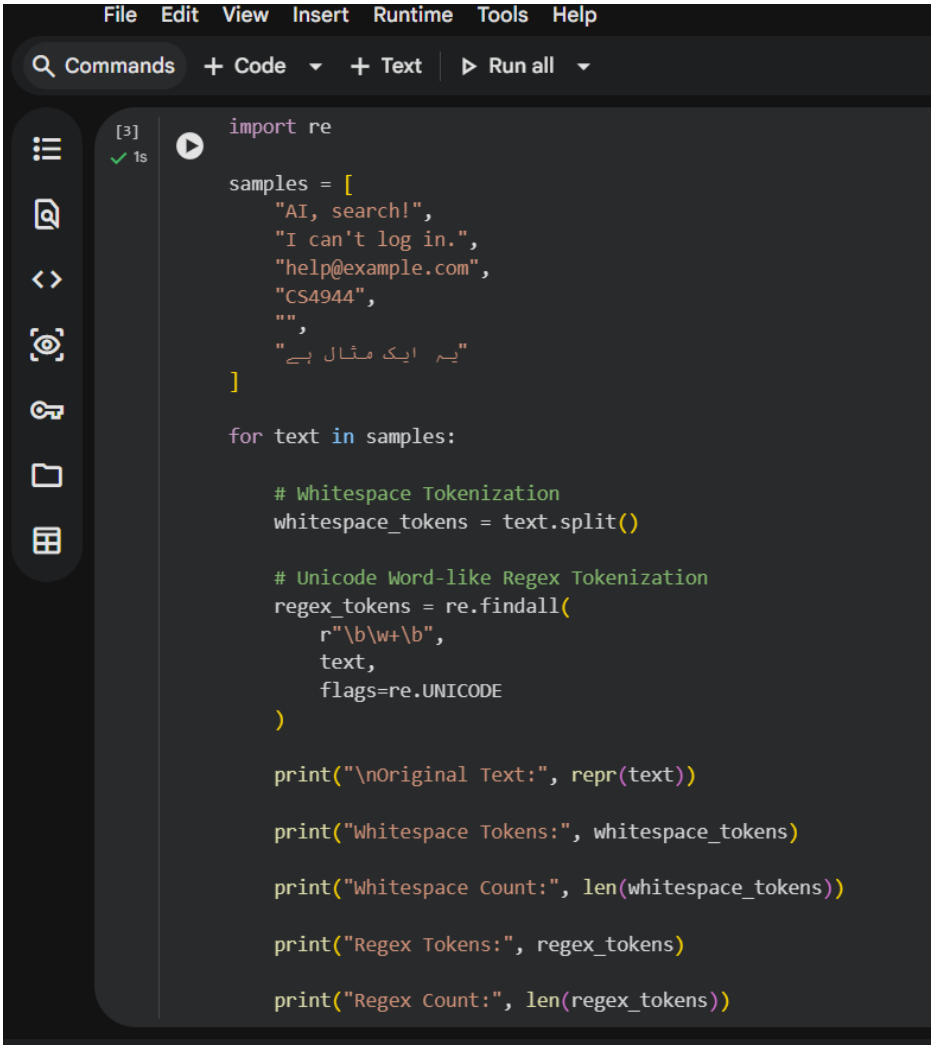
Roll no: 330

Section: BSAI-3A

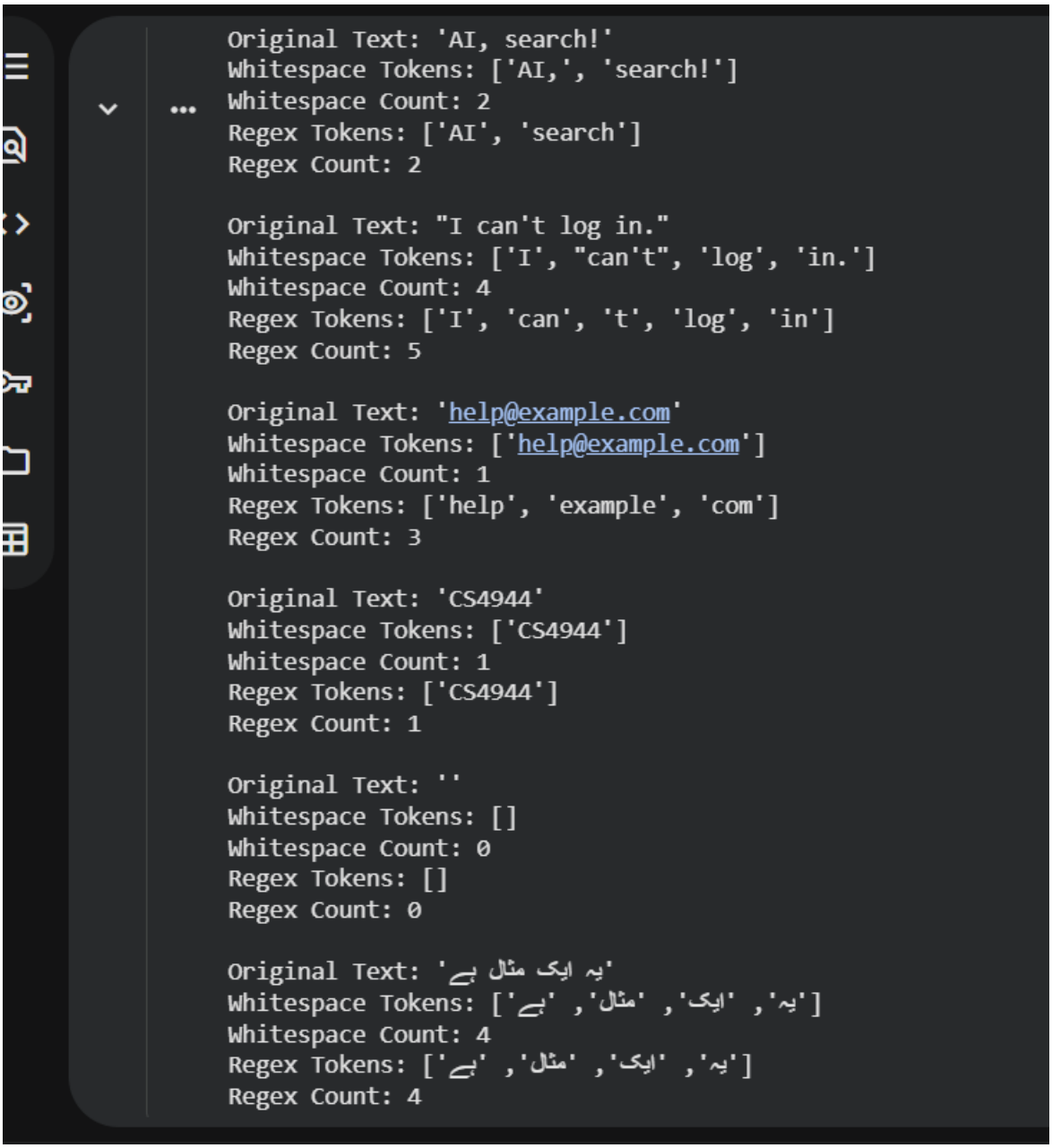
Task 1: Tokenization and a FAQ Chatbot

Tokenization

Code in colab:



Output:



Explanation of Tokenization

Difference 1: Whitespace vs Regex

Whitespace tokenization separates text using spaces. It keeps punctuation attached to words, whereas regex generally separates punctuation and extracts word-like units.

Difference 2: Contractions and Emails

Regex can split contractions such as can't into can and t. It can also split an email address into help, example and com.

Normalization Risk:

Removing punctuation or changing Unicode characters can change the meaning of text, email addresses, identifiers and Urdu words. Regex is a simple teaching method, not a complete multilingual tokenizer.

Part B – FAQ Chatbot

Model Tokenizer

```
[4]
✓ 38s
```

```python
from sentence_transformers import SentenceTransformer

model = SentenceTransformer(
    "sentence-transformers/all-MiniLM-L6-v2"
)

english_examples = [
    "AI, search!",
    "I can't log in."
]

for text in english_examples:

    tokens = model.tokenizer.tokenize(text)

    print("\nOriginal Text:", text)

    print("Model Tokens:", tokens)

    print("Model Token Count:", len(tokens))
```

```
Original Text: AI, search!
Model Tokens: ['ai', ',', 'search', '!']
Model Token Count: 4

Original Text: I can't log in.
Model Tokens: ['i', 'can', "'", 't', 'log', 'in', '.']
Model Token Count: 7
```

FAQ Dataset

```
[6]
✓ 0s
FAQ = [

    {
        "id": "faq:password",
        "question": "How do I reset my password?",
        "answer": "Use the recovery link on the login page.",
        "source": "it_help.txt"
    },

    {
        "id": "faq:submit",
        "question": "Where do I submit my assignment?",
        "answer": "Use the assignment upload page in the learning portal.",
        "source": "assessment.txt"
    },

    {
        "id": "faq:late",
        "question": "Can illness justify a late submission?",
        "answer": "Request an extension with medical evidence before the deadline.",
        "source": "handbook.txt"
    },

    {
        "id": "faq:library",
        "question": "How can I access library books?",
        "answer": "Search the university library catalogue using your student account.",
        "source": "library.txt"
    },
```

Commands + Code + Text ▷ Run all

```
[6]
✓ 0s
    {
        "id": "faq:email",
        "question": "How can I contact IT support?",
        "answer": "Contact the university IT help desk through its official support channel.",
        "source": "it_support.txt"
    }

]

print("Total FAQs:", len(FAQ))

for item in FAQ:
    print(item["id"], "-", item["question"])
```

```
Total FAQs: 5
faq:password - How do I reset my password?
faq:submit - Where do I submit my assignment?
faq:late - Can illness justify a late submission?
faq:library - How can I access library books?
faq:email - How can I contact IT support?
```

TF-IDF Retriever:

[7]
✓ 0s

```python
from sklearn.feature_extraction.text import TfidfVectorizer

class Retriever:

    def __init__(self, records):

        self.records = records

        texts = [
            record["text"]
            for record in records
        ]

        self.vectorizer = TfidfVectorizer()

        self.matrix = self.vectorizer.fit_transform(texts)

    def search(self, query, k=3):

        if not query.strip():
            return []

        query_vector = self.vectorizer.transform([query])

        if query_vector.nnz == 0:
            return []

        scores = (
            query_vector @ self.matrix.T
        ).toarray()[0]
```



```
[7]
✓ 0s

            ranked = sorted(
                range(len(self.records)),
                key=lambda i: (-scores[i], i)
            )

            return [
                (self.records[i], float(scores[i]))
                for i in ranked[:k]
                if scores[i] > 0
            ]


    records = [
        {
            **item,
            "text": item["question"] + " " + item["answer"]
        }
        for item in FAQ
    ]

    engine = Retriever(records)

    print("TF-IDF Retriever created successfully!")

    TF-IDF Retriever created successfully!
```

Main FAQ Chatbot:

[8] ✓ 0s

```python
def reply(engine, query):

    if not query.strip():

        return "Please enter a question."

    # Exact question matching

    for r in FAQ:

        if query.strip().casefold() == r["question"].casefold():

            return (
                r["answer"]
                + " [source: "
                + r["source"]
                + "]"
            )

    # Retrieve suggestions

    hits = engine.search(query)

    if not hits:

        return "No matching FAQ. Please contact the help desk."

    lines = [
        "Possible matches; choose or rephrase your question:"
    ]
```

[8] ✓ 0s

```python
    for r, score in hits:

        lines.append(
            r["question"]
            + f" (similarity {score:.3f})"
        )

    return "\n".join(lines)


print("FAQ Chatbot is ready!")
```

```
FAQ Chatbot is ready!
```

Test Cases:

```
[9]
✓ 0s
print("TEST 1: Exact Question")

print(
    reply(
        engine,
        "How do I reset my password?"
    )
)




print("\nTEST 2: Suggestion")

print(
    reply(
        engine,
        "assignment upload"
    )
)


print("\nTEST 3: Empty Input")

print(
    reply(
        engine,
        ""
    )
)
```

```
[9]
✓ 0s
print("\nTEST 4: Unsupported Question")

print(
    reply(
        engine,
        "volcano geology"
    )
)

TEST 1: Exact Question
Use the recovery link on the login page. [source: it_help.txt]

TEST 2: Suggestion
Possible matches; choose or rephrase your question:
Where do I submit my assignment? (similarity 0.577)

TEST 3: Empty Input
Please enter a question.

TEST 4: Unsupported Question
No matching FAQ. Please contact the help desk.
```

Interactive Chatbot:

```
[10]
✓ 3m
while True:

    query = input("You: ")

    if query.lower().strip() == "exit":

        print("Chatbot stopped.")

        break

    print("Bot:", reply(engine, query))
```

```
You: why do i reset my password
Bot: Possible matches; choose or rephrase your question:
How do I reset my password? (similarity 0.576)
Where do I submit my assignment? (similarity 0.195)
You: stop
Bot: No matching FAQ. Please contact the help desk.
You: exit
Chatbot stopped.
```

Task 2: Chunking and Semantic Suggestions

Three synthetic documents

```
[2]
✓ 0s
corpus = {
    "student_portal": """
The student portal allows students to view their academic information,
attendance records, examination schedules, and assignment announcements.
Every student must log in using a registered university email address
and a secure password. Students should never share their passwords with
other people. If a student forgets a password, the student can use the
password recovery option available on the login page. A verification
code is sent to the registered email address. Students must enter the
correct code before creating a new password.

Assignments must be uploaded before the announced deadline. Late
assignments normally receive a penalty according to the course policy.
However, students with verified medical emergencies may request an
extension from the course coordinator. The request must include valid
medical evidence and should be submitted as soon as possible.

The portal also displays course announcements and updated class
timetables. Students should check announcements regularly because
teachers may change classroom locations or examination schedules.
If the portal becomes unavailable, students should wait and try again
after some time. For urgent academic issues, they can contact the
university support office during working hours. Students are responsible
for checking that their uploaded files are readable and belong to the
correct course. A successful upload message should appear after submission.
The portal keeps a record of submitted assignments for later verification.
Students should save important announcements for future reference.
""",
```

[2]
✓ 0s

```
    "library_access": """
The university library provides books, reference materials, research
journals, and digital learning resources to registered students.
Students must present a valid university identification card when
borrowing physical books. A student can borrow up to three books at
one time for a period of fourteen days. Books must be returned on or
before the due date to avoid late return penalties.

Students may renew a borrowed book once if another student has not
reserved that book. However, reserved books cannot be renewed and
must be returned by the original due date. Students can search for
available books through the library catalogue using a title, author,
or subject keyword. If a book is marked unavailable, students may
place a reservation request through the library desk.

The reading area is open during official library hours. Silence must
be maintained so that other students can study without interruption.
Food and drinks are not allowed near library computers or books.
Students should handle library materials carefully and report damaged
books to the librarian immediately. Lost books must be reported as
soon as possible, and the student may be required to pay replacement
costs according to library policy.

Digital resources can be accessed using university login credentials.
Students should log out after using shared computers to protect their
accounts. The librarian can help students locate research materials
and understand borrowing rules. Library notices may announce changes
in opening hours during examinations and holidays. Students should
check these notices before visiting the library.
""",
```

[2]
✓ 0s

```
the normal support queue. Students should avoid installing unknown
software because it may contain harmful programs.

Support staff may request screenshots of error messages, but students
must hide private information before sharing screenshots. The support
office records each request and provides an identification number.
Students can use this number to check the progress of their request.
If a problem remains unresolved, the request may be escalated to a
senior technician for further investigation. Regular software updates
and safe browsing habits help reduce common technical problems.
"""
}

for doc_id, text in corpus.items():
    print(doc_id, ":", len(text.split()), "words")
```

```
student_portal : 225 words
library_access : 246 words
it_support : 243 words
```

Chunking functions



```
[3]
✓ 0s

import re

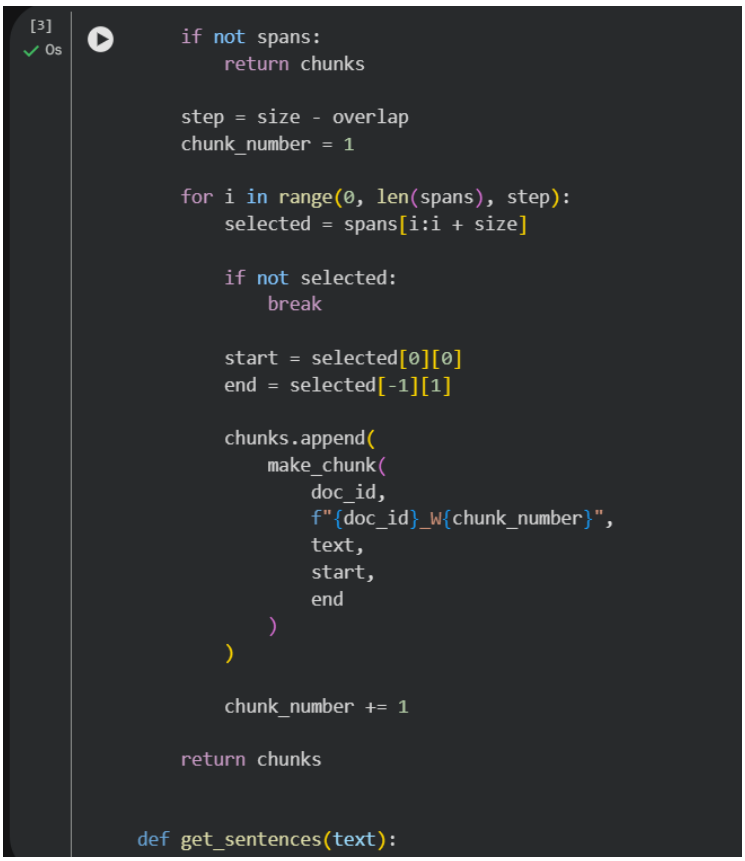
def get_word_spans(text):
    """Return start and end character positions of every word."""
    return [(m.start(), m.end()) for m in re.finditer(r"\S+", text)]


def make_chunk(doc_id, chunk_id, text, start, end):
    """Store exact source text and its character offsets."""
    return {
        "id": chunk_id,
        "doc": doc_id,
        "start": start,
        "end": end,
        "text": text[start:end]
    }


def windows(text, size=40, overlap=0, doc_id="document"):
    """Create fixed-size word windows."""
    if size <= 0:
        raise ValueError("Size must be greater than zero.")

    if overlap < 0 or overlap >= size:
        raise ValueError("Overlap must be between 0 and size - 1.")

    spans = get_word_spans(text)
    chunks = []
```



```
[3]
✓ 0s

    if not spans:
        return chunks

    step = size - overlap
    chunk_number = 1

    for i in range(0, len(spans), step):
        selected = spans[i:i + size]

        if not selected:
            break

        start = selected[0][0]
        end = selected[-1][1]

        chunks.append(
            make_chunk(
                doc_id,
                f"{doc_id}_W{chunk_number}",
                text,
                start,
                end
            )
        )

        chunk_number += 1

    return chunks


def get_sentences(text):
```

```
[3]
✓ 0s
        """Split text using punctuation followed by whitespace."""
        sentences = []

        for match in re.finditer(r"\S.*?(?:[.!?](?=\s|$)|$)", text, re.DOTALL):
            start, end = match.span()
            sentences.append((start, end))

        return sentences


    def sentence_chunks(text, size=40, doc_id="document"):
        """Pack complete sentences within a word budget."""
        if size <= 0:
            raise ValueError("Size must be greater than zero.")

        sentences = get_sentences(text)
        chunks = []
        current = []
        current_words = 0
        chunk_number = 1

        def save_chunk(sentence_group, number):
            start = sentence_group[0][0]
            end = sentence_group[-1][1]

            return make_chunk(
                doc_id,
                f"{doc_id}_S{number}",
                text,
                start,
                end
            )
```

```
[3]
✓ 0s
        for start, end in sentences:
            sentence = text[start:end]
            word_count = len(sentence.split())

            # A sentence longer than the budget needs fallback windows.
            if word_count > size:

                if current:
                    chunks.append(save_chunk(current, chunk_number))
                    chunk_number += 1
                    current = []
                    current_words = 0

                long_sentence = text[start:end]

                fallback_chunks = windows(
                    long_sentence,
                    size=size,
                    overlap=0,
                    doc_id=doc_id
                )

                for fallback in fallback_chunks:
                    absolute_start = start + fallback["start"]
                    absolute_end = start + fallback["end"]

                    chunks.append(
                        make_chunk(
                            doc_id,
                            f"{doc_id}_S{chunk_number}",
                            text,
                            absolute_start,
```

```
[3]
✓ 0s
                        absolute_end
                    )
                )
                chunk_number += 1

            continue

        # Save the current chunk if the next sentence exceeds the budget.
        if current_words + word_count > size and current:
            chunks.append(save_chunk(current, chunk_number))
            chunk_number += 1
            current = []
            current_words = 0

        current.append((start, end))
        current_words += word_count

    if current:
        chunks.append(save_chunk(current, chunk_number))

    return chunks


def make_all_chunks(corpus, strategy):
    """Create chunks for all documents using one strategy."""
    all_chunks = []

    for doc_id, text in corpus.items():

        if strategy == "overlap_0":
            result = windows(text, size=40, overlap=0, doc_id=doc_id)
```

```
[3]
✓ 0s
        elif strategy == "overlap_10":
            result = windows(text, size=40, overlap=10, doc_id=doc_id)

        elif strategy == "sentence":
            result = sentence_chunks(text, size=40, doc_id=doc_id)

        else:
            raise ValueError("Unknown chunking strategy.")

        all_chunks.extend(result)

    return all_chunks


print("Chunking functions created successfully.")

... Chunking functions created successfully.
```

Compare all three chunking strategies

Total chunks

Minimum aur maximum chunk size

Repeated words

```
[4]
✓ 0s
strategies = ["overlap_0", "overlap_10", "sentence"]

# Important rules and exceptions in our documents
labelled_evidence = {
    "student_portal": [
        "Late assignments normally receive a penalty according to the course policy.",
        "However, students with verified medical emergencies may request an extension from the course coordinator."
    ],
    "library_access": [
        "A student can borrow up to three books at one time for a period of fourteen days.",
        "However, reserved books cannot be renewed and must be returned by the original due date."
    ],
    "it_support": [
        "Some applications require approval from the department before installation.",
        "However, urgent security incidents must be reported immediately to the IT office and should not wait for the normal support queue."
    ]
}

results = {}

for strategy in strategies:

    chunks = make_all_chunks(corpus, strategy)
    results[strategy] = chunks

    sizes = [len(c["text"].split()) for c in chunks]

    original_words = sum(len(t.split()) for t in corpus.values())
    chunk_words = sum(sizes)
    repeated_words = chunk_words - original_words
```

```
[4]
✓ 0s
    original_words = sum(len(t.split()) for t in corpus.values())
    chunk_words = sum(sizes)
    repeated_words = chunk_words - original_words

    print("\n" + "=" * 55)
    print("STRATEGY:", strategy)
    print("=" * 55)

    print("Total chunks:", len(chunks))
    print("Minimum chunk size:", min(sizes) if sizes else 0)
    print("Maximum chunk size:", max(sizes) if sizes else 0)
    print("Repeated words:", repeated_words)

    print("\nRule and exception coverage:")

    for doc_id, evidence_list in labelled_evidence.items():

        print("\nDocument:", doc_id)

        for evidence in evidence_list:

            matching_ids = [
                c["id"] for c in chunks
                if c["doc"] == doc_id and evidence in c["text"]
            ]

            print("Evidence:", evidence)
            print("Chunk IDs:", matching_ids if matching_ids else "Not found in one chunk")
```

```
==================================================
STRATEGY: overlap_0
==================================================
Total chunks: 20
Minimum chunk size: 3
Maximum chunk size: 40
Repeated words: 0

Rule and exception coverage:

Document: student_portal
Evidence: Late assignments normally receive a penalty according to the course policy.
Chunk IDs: Not found in one chunk
Evidence: However, students with verified medical emergencies may request an extension from the course coordinator.
Chunk IDs: Not found in one chunk

Document: library_access
Evidence: A student can borrow up to three books at one time for a period of fourteen days.
Chunk IDs: Not found in one chunk
Evidence: However, reserved books cannot be renewed and must be returned by the original due date.
Chunk IDs: Not found in one chunk

Document: it_support
Evidence: Some applications require approval from the department before installation.
Chunk IDs: Not found in one chunk
Evidence: However, urgent security incidents must be reported immediately to the IT office and should not wait for the normal support queue.
Chunk IDs: Not found in one chunk
```

```
==================================================
STRATEGY: overlap_10
==================================================
Total chunks: 26
Minimum chunk size: 3
Maximum chunk size: 40
Repeated words: 219

Rule and exception coverage:

Document: student_portal
Evidence: Late assignments normally receive a penalty according to the course policy.
Chunk IDs: Not found in one chunk
Evidence: However, students with verified medical emergencies may request an extension from the course coordinator.
Chunk IDs: Not found in one chunk

Document: library_access
Evidence: A student can borrow up to three books at one time for a period of fourteen days.
Chunk IDs: Not found in one chunk
Evidence: However, reserved books cannot be renewed and must be returned by the original due date.
Chunk IDs: Not found in one chunk

Document: it_support
Evidence: Some applications require approval from the department before installation.
Chunk IDs: Not found in one chunk
Evidence: However, urgent security incidents must be reported immediately to the IT office and should not wait for the normal support queue.
Chunk IDs: Not found in one chunk

==================================================
STRATEGY: sentence
==================================================
Total chunks: 22
Minimum chunk size: 19
Maximum chunk size: 40
Repeated words: 0
```

```
==================================================
STRATEGY: sentence
==================================================
Total chunks: 22
Minimum chunk size: 19
Maximum chunk size: 40
Repeated words: 0

Rule and exception coverage:

Document: student_portal
Evidence: Late assignments normally receive a penalty according to the course policy.
Chunk IDs: Not found in one chunk
Evidence: However, students with verified medical emergencies may request an extension from the course coordinator.
Chunk IDs: Not found in one chunk

Document: library_access
Evidence: A student can borrow up to three books at one time for a period of fourteen days.
Chunk IDs: Not found in one chunk
Evidence: However, reserved books cannot be renewed and must be returned by the original due date.
Chunk IDs: Not found in one chunk

Document: it_support
Evidence: Some applications require approval from the department before installation.
Chunk IDs: Not found in one chunk
Evidence: However, urgent security incidents must be reported immediately to the IT office and should not wait for the normal support queue.
Chunk IDs: Not found in one chunk
```

Check character offsets and original evidence

```
[5]
✓ 0s
```

```python
print("Checking chunk source offsets...")

for strategy, chunks in results.items():

    for chunk in chunks:
        original_text = corpus[chunk["doc"]]

        extracted_text = original_text[
            chunk["start"]:chunk["end"]
        ]

        assert extracted_text == chunk["text"]

        assert chunk["start"] < chunk["end"]

    print(strategy, "All offsets verified successfully.")

print("\nAll chunk source checks passed.")
```

```
Checking chunk source offsets...
overlap_0 All offsets verified successfully.
overlap_10 All offsets verified successfully.
sentence All offsets verified successfully.

All chunk source checks passed.
```

Edge case testing

1. Empty text
2. Document shorter than 40 words
3. Invalid overlap
4. Sentence longer than 40 words

[6]
✓ 0s

```
print("TEST 1: Empty text")

empty_result = windows("", size=40, overlap=0)
print("Chunks:", empty_result)
print("Passed:", empty_result == [])


print("\nTEST 2: Short document")

short_text = "Artificial Intelligence helps students learn new concepts."

short_result = windows(
    short_text,
    size=40,
    overlap=0
)

print("Number of chunks:", len(short_result))
print("Chunk text:", short_result[0]["text"])
print("Passed:", len(short_result) == 1)


print("\nTEST 3: Invalid overlap")

try:
    windows("This is a test document.", size=40, overlap=40)

except ValueError as error:
    print("Error handled:", error)
    print("Passed: True")
```

```
[6]
✓ 0s
try:
    windows("This is a test document.", size=40, overlap=40)

except ValueError as error:
    print("Error handled:", error)
    print("Passed: True")


print("\nTEST 4: Long sentence")

long_sentence = (
    "Students should carefully check their university portal every day "
    "because important announcements, examination schedules, assignment "
    "deadlines, classroom changes, academic notices, registration updates, "
    "and other university information may be published there without "
    "individual messages being sent to every student."
)

long_result = sentence_chunks(
    long_sentence,
    size=40,
    doc_id="long_test"
)

print("Number of chunks:", len(long_result))

for chunk in long_result:
    print(chunk["id"], "Words:", len(chunk["text"].split()))
    print(chunk["text"])

print("Passed:", all(len(c["text"].split()) <= 40 for c in long_result))
```

```
TEST 1: Empty text
Chunks: []
Passed: True

TEST 2: Short document
Number of chunks: 1
Chunk text: Artificial Intelligence helps students learn new concepts.
Passed: True

TEST 3: Invalid overlap
Error handled: Overlap must be between 0 and size - 1.
Passed: True

TEST 4: Long sentence
Number of chunks: 1
long_test_S1 Words: 38
Students should carefully check their university portal every day because important announcements, examination schedules, assignment deadlines, classroom changes, academic no
Passed: True
```

TF-IDF vs Semantic Embeddings

```
[ ]
from sklearn.feature_extraction.text import TfidfVectorizer
from sentence_transformers import SentenceTransformer
import numpy as np
import time

queries = [
    "What can a student do if illness causes a late assignment?",
    "Can a student extend the return date of a reserved library book?",
    "What should users do when an urgent security problem occurs?"
]

all_chunks = results["overlap_10"]
chunk_texts = [c["text"] for c in all_chunks]

print("Loading sentence embedding model...")

start_time = time.perf_counter()

model = SentenceTransformer(
    "sentence-transformers/all-MiniLM-L6-v2",
    device="cpu"
)

print("Model loading seconds:", round(time.perf_counter() - start_time, 2))


# TF-IDF
vectorizer = TfidfVectorizer()
```

```
[ ]
chunk_vectors = vectorizer.fit_transform(chunk_texts)
query_vectors = vectorizer.transform(queries)

tfidf_scores = (query_vectors @ chunk_vectors.T).toarray()


# Real sentence embeddings
chunk_embeddings = model.encode(
    chunk_texts,
    normalize_embeddings=True
)

query_embeddings = model.encode(
    queries,
    normalize_embeddings=True
)

embedding_scores = query_embeddings @ chunk_embeddings.T


# Display top 3 results
for q_index, query in enumerate(queries):

    print("\n" + "=" * 65)
    print("QUERY:", query)
    print("=" * 65)

    for method, scores in [
        ("TF-IDF", tfidf_scores),
        ("SEMANTIC EMBEDDINGS", embedding_scores)
    ]:
```

```
[ ]
        print("\nMethod:", method)

        top_indices = np.argsort(
            -scores[q_index],
            kind="stable"
        )[:3]

        for rank, index in enumerate(top_indices, start=1):

            chunk = all_chunks[index]

            print("\nRank:", rank)
            print("Chunk ID:", chunk["id"])
            print("Document:", chunk["doc"])
            print("Score:", round(float(scores[q_index][index]), 4))
            print("Original evidence:")
            print(chunk["text"])
```

Final verification

```
print("TASK 2 FINAL CHECK")

print("Documents:", len(corpus))
print("Strategies tested:", len(results))

for strategy, chunks in results.items():
    print(strategy, ":", len(chunks), "chunks")

print("\nQueries tested:", len(queries))
print("Retrieval methods: TF-IDF and Sentence Embeddings")

print("\nTask 2 execution completed.")
```

```
TASK 2 FINAL CHECK
Documents: 3
Strategies tested: 3
overlap_0 : 20 chunks
overlap_10 : 26 chunks
sentence : 22 chunks

Queries tested: 3
Retrieval methods: TF-IDF and Sentence Embeddings

Task 2 execution completed.
```

Difference between the three strategies

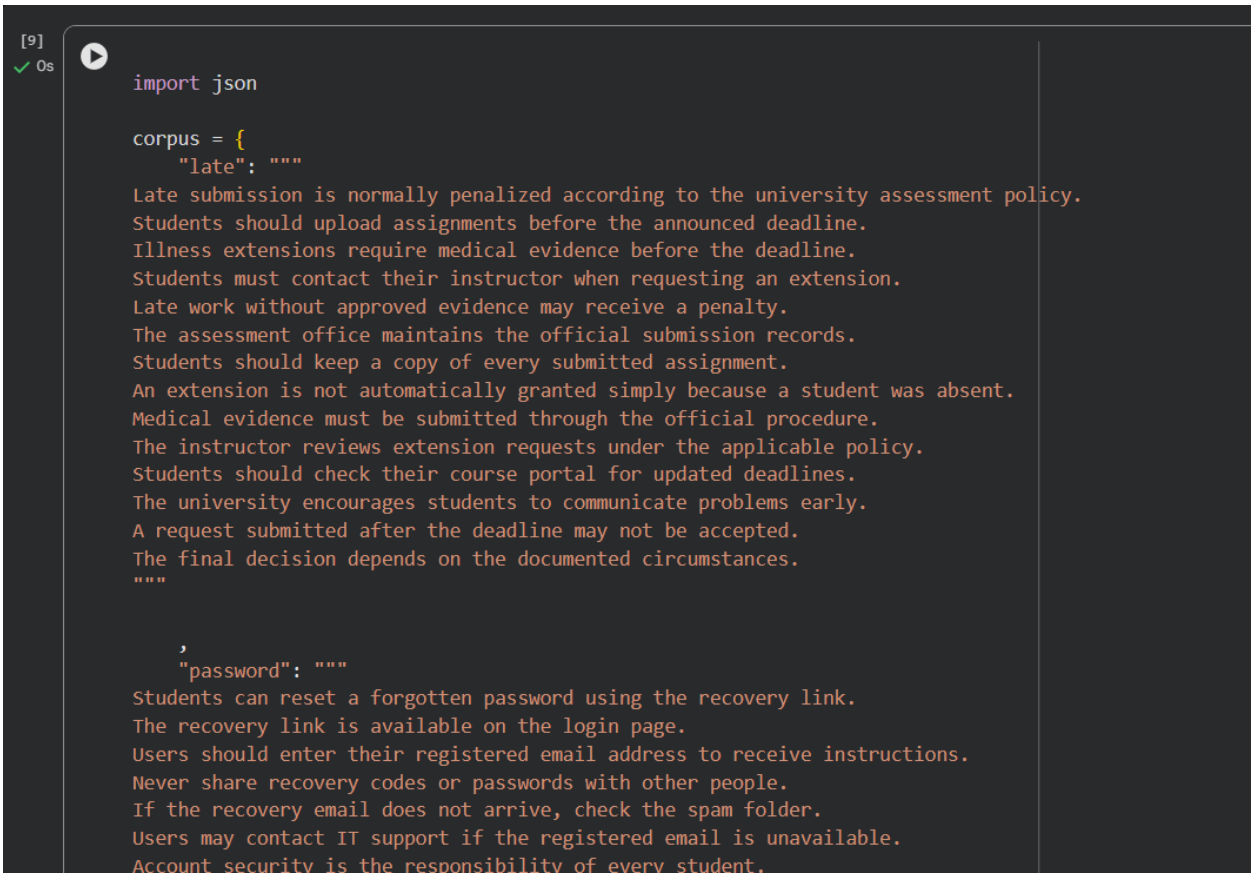
Strategy	Explanation
Overlap 0	Divides text into 40-word chunks without repetition.

Overlap 10 Repeats 10 words between consecutive chunks to preserve context.

Sentence-aware Tries to keep complete sentences together within a 40-word budget.

Task 3: Advanced – Evaluate a Document Retrieval Chatbot

dataset



```
[9]
✓ 0s
import json

corpus = {
    "late": """
Late submission is normally penalized according to the university assessment policy.
Students should upload assignments before the announced deadline.
Illness extensions require medical evidence before the deadline.
Students must contact their instructor when requesting an extension.
Late work without approved evidence may receive a penalty.
The assessment office maintains the official submission records.
Students should keep a copy of every submitted assignment.
An extension is not automatically granted simply because a student was absent.
Medical evidence must be submitted through the official procedure.
The instructor reviews extension requests under the applicable policy.
Students should check their course portal for updated deadlines.
The university encourages students to communicate problems early.
A request submitted after the deadline may not be accepted.
The final decision depends on the documented circumstances.
"""


    ,
    "password": """
Students can reset a forgotten password using the recovery link.
The recovery link is available on the login page.
Users should enter their registered email address to receive instructions.
Never share recovery codes or passwords with other people.
If the recovery email does not arrive, check the spam folder.
Users may contact IT support if the registered email is unavailable.
Account security is the responsibility of every student.
```

```
[9]
✓ 0s
        "doc":"assignment",
        "evidence":"Assignments must be uploaded through the learning portal.",
        "split":"test"},

    {"query":"What documentation is needed if I was sick?",
     "doc":"late",
     "evidence":"Illness extensions require medical evidence before the deadline.",
     "split":"test"},

    {"query":"What are tomorrow's flight ticket prices?",
     "doc":None,
     "evidence":None,
     "split":"test"},

    {"query":"Who won yesterday's football match?",
     "doc":None,
     "evidence":None,
     "split":"test"}
]

print("Documents:", len(corpus))
print("Queries:", len(queries))
print("Development:", sum(q["split"]=="dev" for q in queries))
print("Testing:", sum(q["split"]=="test" for q in queries))

for q in queries:
    if q["doc"] is not None:
        assert q["evidence"] in corpus[q["doc"]]

print("Dataset validation passed!")
```

```
Documents: 5
Queries: 10
Development: 4
Testing: 6
Dataset validation passed!
```

Chunking functions

[12] ✓ 0s

```python
import re

def word_spans(text):
    return [(m.start(), m.end()) for m in re.finditer(r"\S+", text)]

def windows(text, size=40, overlap=10, offset=0):
    if size <= 0 or not 0 <= overlap < size:
        raise ValueError("Invalid size or overlap")

    spans = word_spans(text)
    result = []

    step = size - overlap

    for i in range(0, len(spans), step):
        j = min(i + size, len(spans))

        result.append((
            offset + spans[i][0],
            offset + spans[j-1][1]
        ))

        if j == len(spans):
            break

    return result

def sentence_chunks(text, size=40):
    if size <= 0:
        raise ValueError("Size must be positive")
```

[12] ✓ 0s

```python
def make_chunks(docs, strategy):
    records = []

    for doc_id, text in docs.items():

        if strategy == "sentence":
            spans = sentence_chunks(text, 40)
        else:
            spans = windows(text, 40, int(strategy))

        for i, (a, b) in enumerate(spans):
            records.append({
                "id": f"{doc_id}:{i}",
                "doc": doc_id,
                "start": a,
                "end": b,
                "text": text[a:b]
            })

    return records

print("Chunking functions ready.")
```

```
Chunking functions ready.
```

Test all chunking strategies and offsets

[13]
✓ 0s

```python
strategies = ["0", "10", "sentence"]

chunks_by_strategy = {}

for strategy in strategies:

    chunks = make_chunks(corpus, strategy)
    chunks_by_strategy[strategy] = chunks

    sizes = [len(c["text"].split()) for c in chunks]

    original_words = sum(len(t.split()) for t in corpus.values())
    emitted_words = sum(sizes)
    repeated_words = emitted_words - original_words

    print("\nStrategy:", strategy)
    print("Total chunks:", len(chunks))
    print("Minimum words:", min(sizes))
    print("Maximum words:", max(sizes))
    print("Repeated word instances:", repeated_words)

    # Verify exact source offsets
    for c in chunks:
        assert corpus[c["doc"]][c["start"]:c["end"]] == c["text"]

print("\nAll offsets verified.")
```

```
Strategy: 0
Total chunks: 16
Minimum words: 7
Maximum words: 40
Repeated word instances: 0

Strategy: 10
Total chunks: 20
Minimum words: 14
Maximum words: 40
Repeated word instances: 150

Strategy: sentence
Total chunks: 16
Minimum words: 18
Maximum words: 39
Repeated word instances: 0

All offsets verified.
```

Edge-case testing



```
[14]
assert windows("") == []
print("Empty text test passed")

assert len(windows("short document", 40, 0)) == 1
print("Short document test passed")


try:
    windows("a b c", 40, 40)
except ValueError:
    print("Invalid overlap test passed")

long_sentence = " ".join(["word"] * 95) + "."

long_chunks = sentence_chunks(long_sentence, 40)

assert all(len(long_sentence[a:b].split()) <= 40
           for a,b in long_chunks)

print("Long sentence test passed")
print("Long sentence chunks:", len(long_chunks))

...
Empty text test passed
Short document test passed
Invalid overlap test passed
Long sentence test passed
Long sentence chunks: 3
```

TF-IDF vs real semantic embeddings

[15]
✓ 5s

```python
import numpy as np
import pandas as pd
import time

from sklearn.feature_extraction.text import TfidfVectorizer
from sentence_transformers import SentenceTransformer

# Measure model loading separately
start = time.perf_counter()

model = SentenceTransformer(
    "sentence-transformers/all-MiniLM-L6-v2"
)

model_load_time = time.perf_counter() - start

print("Model load time:", model_load_time, "seconds")

query_texts = [q["query"] for q in queries]

logs = []

for strategy in strategies:

    chunks = chunks_by_strategy[strategy]
    texts = [c["text"] for c in chunks]

    # Audit model token lengths
    lengths = [
        len(model.tokenizer(t, truncation=False)["input_ids"])
```

[15]
✓ 5s

```python
        Q = model.encode(
            query_texts,
            normalize_embeddings=True
        )

    encoding_time = time.perf_counter() - start

    start = time.perf_counter()

    if method == "tfidf":
        scores = (Q @ C.T).toarray()
    else:
        scores = Q @ C.T

    search_time = time.perf_counter() - start

    print("\nMethod:", method)
    print("Encoding time:", encoding_time)
    print("Search time:", search_time)

    for split in ["dev", "test"]:

        indices = [
            i for i,q in enumerate(queries)
            if q["split"] == split
        ]

        hits = []
        reciprocal_ranks = []

        for qi in indices:
```

[15] ✓ 5s

```
print("\nMethod:", method)
print("Encoding time:", encoding_time)
print("Search time:", search_time)

for split in ["dev", "test"]:

    indices = [
        i for i,q in enumerate(queries)
        if q["split"] == split
    ]

    hits = []
    reciprocal_ranks = []

    for qi in indices:

        q = queries[qi]
        row = scores[qi]

        top3 = np.argsort(-row, kind="stable")[:3]

        relevant_rank = None

        if q["doc"] is not None:

            for rank, ci in enumerate(top3, 1):

                c = chunks[ci]

                if (c["doc"] == q["doc"]
                    and q["evidence"] in c["text"]):
```

[15] ✓ 5s

```
                    and q[ evidence ] in c[ text ]):

                    relevant_rank = rank
                    break

            hits.append(relevant_rank is not None)

            reciprocal_ranks.append(
                1/relevant_rank if relevant_rank else 0
            )

    hit3 = np.mean(hits) if hits else None
    mrr3 = np.mean(reciprocal_ranks) if hits else None

    print(
        strategy, method, split,
        "Hit@3:", hit3,
        "MRR@3:", mrr3
    )

# Log top 3 results for every query
for qi, q in enumerate(queries):

    top3 = np.argsort(
        -scores[qi], kind="stable"
    )[:3]

    for rank, ci in enumerate(top3, 1):

        c = chunks[ci]

        record = {
```

[15]
✓ 5s

```
            for rank, ci in enumerate(top3, 1):

                c = chunks[ci]

                record = {
                    "strategy": strategy,
                    "method": method,
                    "query": q["query"],
                    "split": q["split"],
                    "answerable": q["doc"] is not None,
                    "rank": rank,
                    "score": float(scores[qi][ci]),
                    "chunk_id": c["id"],
                    "doc": c["doc"],
                    "start": c["start"],
                    "end": c["end"],
                    "text": c["text"]
                }

                logs.append(record)

                print(
                    "Rank:", rank,
                    "ID:", c["id"],
                    "Score:", round(float(scores[qi][ci]), 3),
                    "Evidence:", c["text"]
                )

print("\nRetrieval evaluation complete.")
```

```
The university encourages students to communicate problems early.
```

```
[15]
✓ 5s
print("\nRetrieval evaluation complete.")

Do not remove cartridges while the printer is
Rank: 3 ID: printer:2 Score: 0.585 Evidence: operating.
Contact IT support if basic troubleshooting fails.
The support team may inspect the printer hardware.
Record the printer model when reporting a problem.
Rank: 1 ID: password:0 Score: 0.559 Evidence: Students can reset a forgotten password using the recovery
The recovery link is available on the login page.
Users should enter their registered email address to receive instructions.
Never share recovery codes or passwords with other people.
If the
Rank: 2 ID: password:1 Score: 0.481 Evidence: recovery email does not arrive, check the spam folder.
Users may contact IT support if the registered email is unavailable.
Account security is the responsibility of every student.
A password should be strong and difficult to guess.
The system may
Rank: 3 ID: password:2 Score: 0.408 Evidence: temporarily lock accounts after repeated failed attempts.
Students should follow the official recovery procedure.
Recovery codes must be kept private and stored safely.
IT support never asks users to reveal their password.
Rank: 1 ID: assignment:0 Score: 0.445 Evidence: Assignments must be uploaded through the learning portal
Students should submit the final assignment file as a PDF.
The portal displays a confirmation after successful submission.
Students should verify that the uploaded file opens correctly.
A wrong or corrupted file
Rank: 2 ID: assignment:1 Score: 0.373 Evidence: may not be considered a valid submission.
The submission deadline is displayed on the course page.
Students should keep the submission confirmation for their records.
If the portal is unavailable, contact the instructor immediately.
Email submission is not accepted unless
Rank: 3 ID: late:2 Score: 0.361 Evidence: the official procedure.
The instructor reviews extension requests under the applicable policy.
Students should check their course portal for updated deadlines.
The university encourages students to communicate problems early.
```

```
Rank: 2 ID: printer:1 Score: -0.029 Evidence: If the problem continues, restart the printer.
Users should verify the selected printer in the computer settings.
A printer may produce blank pages when ink is empty.
The cleaning utility can help clear blocked print heads.

Rank: 3 ID: late:0 Score: -0.031 Evidence:
Late submission is normally penalized according to the university assessment policy.
Students should upload assignments before the announced deadline.
Illness extensions require medical evidence before the deadline.
Students must contact their instructor when requesting an extension.

Rank: 1 ID: password:1 Score: 0.057 Evidence: If the recovery email does not arrive, check the spam folder.
Users may contact IT support if the registered email is unavailable.
Account security is the responsibility of every student.
A password should be strong and difficult to guess.

Rank: 2 ID: assignment:1 Score: 0.055 Evidence: A wrong or corrupted file may not be considered a valid submission.
The submission deadline is displayed on the course page.
Students should keep the submission confirmation for their records.
If the portal is unavailable, contact the instructor immediately.

Rank: 3 ID: late:2 Score: 0.047 Evidence: Medical evidence must be submitted through the official procedure.
The instructor reviews extension requests under the applicable policy.
Students should check their course portal for updated deadlines.
The university encourages students to communicate problems early.


Retrieval evaluation complete.
```

Export retrieval log to CSV

```
[16]
✓ 0s
```

```python
df = pd.DataFrame(logs)

df.to_csv("retrieval_log.csv", index=False)

print("CSV file created successfully.")
print("Total logged records:", len(df))

display(df.head(10))
```

```
CSV file created successfully.
Total logged records: 180
```

	strategy	method	query	split	answerable	rank	score	chunk_id	doc	start	end	text
0	0	tfidf	What proof is needed for an illness extension?	dev	True	1	0.265177	late:0	late	1	312	Late submission is normally penalized accordin...
1	0	tfidf	What proof is needed for an illness extension?	dev	True	2	0.174446	late:1	late	313	590	evidence may receive a penalty.\nThe assessmen...
2	0	tfidf	What proof is needed for an illness extension?	dev	True	3	0.123532	assignment:1	assignment	271	543	may not be considered a valid submission.\nThe...
3	0	tfidf	How can I recover my account password?	dev	True	1	0.211790	password:1	password	257	504	recovery email does not arrive, check the spam...
4	0	tfidf	How can I recover my account password?	dev	True	2	0.142003	password:0	password	1	256	Students can reset a forgotten password using ...
5	0	tfidf	How can I recover my account password?	dev	True	3	0.085322	password:2	password	505	727	temporarily lock accounts after repeated faile...
6	0	tfidf	Where should coursework be submitted?	dev	True	1	0.338155	late:1	late	313	590	evidence may receive a penalty.\nThe assessmen...
7	0	tfidf	Where should coursework be submitted?	dev	True	2	0.213048	late:2	late	591	880	the official procedure.\nThe instructor review...
8	0	tfidf	Where should coursework be submitted?	dev	True	3	0.162778	contractor:1	contractor	307	585	approval from the project manager.\nCompany in...
9	0	tfidf	Are contractors allowed to work from home?	dev	True	1	0.312813	contractor:0	contractor	1	306	Contractors can work remotely only if the sign...

Limitations

The dataset contains synthetic documents and a limited number of queries. The sentence splitter may incorrectly split abbreviations or fail to identify sentence boundaries in some languages. TF-IDF may fail on paraphrases with little lexical overlap. Embeddings can retrieve semantically related but incorrect evidence. Conflicting documents may also produce misleading results.

Conclusion

This task demonstrates how chunking, retrieval methods, and evaluation metrics affect document retrieval performance. The system maintains source offsets and evidence traceability. The development results are used to select a configuration, while held-out queries evaluate its generalization. The chatbot provides evidence for confirmation rather than generating unsupported answers.